

PICOGPT

Transparentan Transformer baziran na NumPy/CuPy sa PyTorch referentnim modelom

SV8/2023 Djordje Vujanovic, SV45/2023 Maksim Vasic



Svrha projekta

PicoGPT je edukativni Transformer napravljen od nule - ista arhitektura koja pokreće GPT modele, samo u mnogo manjem i razumljivijem obliku.



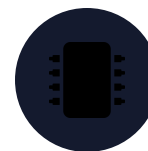
Potpuna transparentnost

Svaka matematička operacija - i unapred (forward) i unazad (backward, tj. gradijenti) - napisana je eksplicitno u NumPy/CuPy kodu. Ništa nije skriveno iza gotovih biblioteka, pa se mogu direktno posmatrati gradijenti i mape pažnje (attention).



Edukativna jasnoća

Kod je namerno razbijen na male, dobro dokumentovane module: tokenizer, učitavač podataka, slojevi modela i petlja za trening. Svaki deo se može izučavati odvojeno, korak po korak.



Pogodnost za slabiji hardver

Osnovna NumPy/CuPy verzija cilja na CPU/GPU uređaje sa ograničenom memorijom. Oracle PyTorch verzija) služi kao referenca za proveru da li NumPy implementacija daje identične rezultate.

Struktura repozitorijuma

pico-gpt/ · organizacija projekta na najvišem nivou



`data_handling/`

Tokenizacija i strimovani pipeline za podatke



`data/`

Generisani delovi (chunk) podataka (pravi ih ChunkerLoader)



`layers/`

Rezervisano za čiste NumPy/CuPy slojeve



`pico_gpt_oracle.pth`

Sačuvani (istrenirani) model (iz oracle/main.py)



`oracle/`

PyTorch referentna implementacija za proveru gradijenata



`main.py`

Ulazna tacka za inference



`notebooks/`

Jupyter beležnice za eksperimentisanje



`pyproject.toml / README.md`

Metapodaci projekta i opšti opis

Pipeline za obradu podataka

Tri modula pretvaraju sirov tekst u gotove trening-pakete (batch-eve)



01

`tokenizer.py`

CharacterTokenizer

Pretvara sirov UTF-8 tekst u niz celobrojnih ID-jeva (svaki karakter dobija svoj broj) i nazad. Model 'ne razume' slova - samo brojeve.



02

`chunker_loader.py`

ChunkerLoader

Strimuje TinyStories skup podataka i deli ga na fajlove fiksne veličine na disku, umesto da sve učitava odjednom u memoriju.



03

`batch_loader.py`

DynamicBatchLoader

Učitava delove (chunk-ove) u RAM i vraća nasumične PyTorch pakete spremne za trening modela.

Tokenizer i ChunkerLoader



CharacterTokenizer

- Azbuka (alfabet): razmaci, cifre, velika/mala slova, interpunkcija i unicode navodnici.
- `encode(text)` → lista ID-jeva tokena; nepoznati karakteri se mapiraju na 0.
- `decode(ids)` → tekst; nepoznat ID se prikazuje kao razmak.
- Zašto na nivou karaktera? Nije potreban spoljašnji tokenizer, a svaka dimenzija predstavlja tačno jedan vidljiv karakter - potpuno razumljivo, ali cena je duži niz tokena za istu količinu teksta (otprilike 5x duže sekvence nego kod sub-word tokenizacije).



ChunkerLoader

Strimuje roneneldan/TinyStories sa HuggingFace-a i upisuje ga na disk u fiksnim delovima - bez učitavanja celog skupa u memoriju.

Ključni parametri

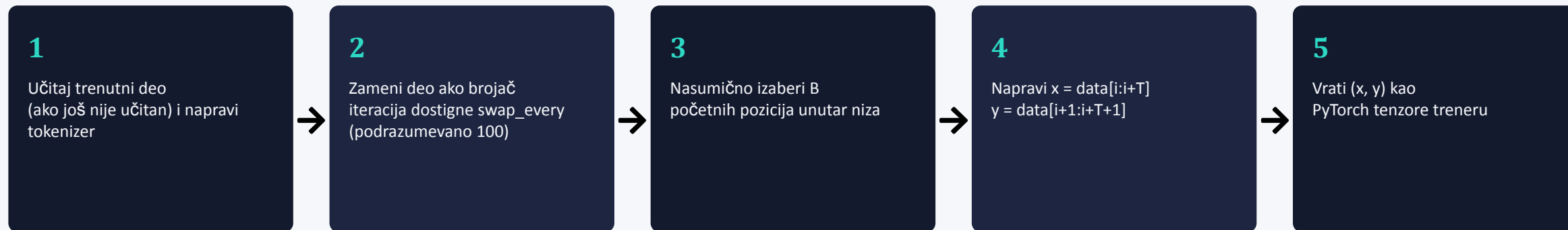
- `stories_per_chunk` (podrazumevano 10.000) - broj priča po .txt fajlu
- `max_chunks` (podrazumevano 10) - ograničava broj delova radi lakšeg demo treninga

Tok rada

1. Strimuje trening deo skupa (`streaming=True`)
2. Deli svaku priču po novom redu; zadržava nepravne linije
3. Upisuje nakupljene linije u `train_chunk_{i}.txt`
4. Ponavlja dok se ne napiše `max_chunks` fajlova
5. Upisuje prvih 2.000 primera za validaciju u `tiny_stories_val.txt`

DynamicBatchLoader

Pravi mini-pakete za trening jezičkog modela, uz povremenu zamenu delova podataka radi uštede memorije.



Interno stanje

- `chunk_files['train'] / ['val']` - sortirane liste fajlova
- `current_chunk_idx` - indeks dela trenutno u memoriji
- `loaded_data` - NumPy int32 niz ID-jeva tokena
- `char_to_int` - jedinstvena mapa tokenizera, deljena između split-ova

Zašto ovakav dizajn?

- Efikasnost memorije - u RAM-u je uvek samo jedan deo podataka
- Statistička raznovrsnost - povremena zamena sprečava preprilagođavanje (overfitting) jednom delu teksta
- Deterministički rečnik - napravljen jednom pa zamrznut, radi doslednosti

Arhitektura modela



Ključne komponente

Četiri modula u oracle/ implementiraju standardnu “moderni LLM” recepturu



`normalization.py`

RMSNorm

Normalizuje svaki vektor osobina po njegovom root-mean-square (koren srednjeg kvadrata), pa ga skalira naučenim faktorom. Jednostavnije od LayerNorm-a jer ne oduzima srednju vrednost - brže, a podjednako efikasno.



`activation.py`

SwiGLU

“Gated” feed-forward mreža: rezultat se dobija kao $\text{SiLU}(\text{kapija}) \odot \text{vrednost}$ - jedan deo mreže odlučuje 'koliko propustiti' drugog dela. Skrivena dimenzija je $2 \times d_{\text{model}}/3$, da broj operacija ostane sličan običnom $4 \times \text{FFN}$ -u.



`position.py`

RoPE

Rotaciono pozicijsko kodiranje - umesto da doda poziciju kao broj, rotira Q/K vektore za ugao koji zavisi od pozicije tokena. Odlično generalizuje na duže nizove i praktično ne dodaje računski trošak.



`cqa.py`

CausalGQABlock

Attention sa više glava (multi-head), sa RoPE primenjenim na Q/K, kauzalnom maskom (`torch.tril`) koja sprečava 'gledanje u budućnost', i skaliranim skalarnim proizvodom. Pripremljen za buduće smanjenje broja K/V glava (prava GQA).

Kako funkcioniše Attention?

Srce Transformera: svaki token “pita” ostale tokene koliko su mu relevantni



Q - Query (upit)

Šta ovaj token traži? Predstavlja pitanje koje token postavlja ostatku rečenice.



K - Key (ključ)

Šta ovaj token nudi? Koristi se da se izmeri koliko dobro odgovara nečijem upitu.



V - Value (vrednost)

Sadržaj koji se zapravo prenosi dalje, kada je 'poklapanje' Q i K jako.

Postupak, korak po korak

1. Q, K, V se dobijaju linearnim projekcijama ulaza x , pa se dele na n_heads “glava” koje gledaju tekst iz različitih uglova.
2. RoPE rotira Q i K prema poziciji tokena, tako da model zna redosled reči.
3. Računaju se “scores” = $(Q \cdot K^T) / \sqrt{head_dim}$ - koliko je svaki token relevantan za svaki drugi.
4. Kauzalna maska postavlja $-\infty$ na sve “buduće” pozicije - token ne sme videti reči koje tek dolaze.
5. Softmax pretvara score-ove u težine (verovatnoće) koje se zbrajaju u 1.
6. Konačni izlaz = težinski zbir V vrednosti - svaki token “upija” informacije od relevantnih prethodnika.

Trening petlja - detaljnije

Šta se tačno dešava u svakom koraku optimizacije (Runner.train())



1. Forward Pass

Ulazni tokeni x prolaze kroz model i daju logits - “ocene” za svaki mogući sledeći karakter, za svaku poziciju u nizu.



2. Računanje gubitka (loss)

Cross-entropy poredi predviđene logits sa stvarnim sledećim karakterom y . Manji gubitak = bolje predviđanje.



3. Backward Pass

`loss.backward()` računa gradijent - za svaki parametar, u kom pravcu i koliko ga promeniti da bi se gubitak smanjio.



4. Gradient Clipping

Gradijenti se ograničavaju na `max_norm=1.0` kako nagli “skokovi” ne bi destabilizovali trening.



5. Optimizer Step

AdamW ažurira sve težine modela na osnovu gradijenata, brzine učenja (`lr=1e-3`) i weight decay-a (`0.01`).



6. Evaluacija (povremeno)

Na svakih `eval_interval` (200) koraka: meri se gubitak na train/val skupu i generiše kratak uzorak teksta radi kvalitativne provere napretka.

PicoGPT - kompletan model i njegovi parametri

Forward tok: `x = x + Attn(RMSNorm(x)); x = x + FFN(RMSNorm(x))` - po sloju, pa final norm + LM head

Podrazumevana konfiguracija (oracle/main.py)

<code>vocab_size</code>	≈ 65 (skup karaktera)
<code>d_model</code>	256
<code>n_layers</code>	4
<code>n_heads</code>	$8 \rightarrow \text{head_dim} = 32$
<code>block_size (T)</code>	128
<code>oblik batch-a (B,T)</code>	(32, 128)
<code>oblik embeddinga</code>	(32, 128, 256)
<code>Q/K/V po glavi</code>	(32, 8, 128, 32)

Zašto baš ova konfiguracija?

- Dovoljno mala da se brzo trenira na CPU/GPU-u za demo, a dovoljno velika da pokaže zanimljivo jezičko ponašanje na TinyStories skupu.
- $d_{\text{ffn}} = \frac{2}{3} \times d_{\text{model}}$ prati efikasna podešavanja popularizovana kod LLaMA modela.
- LM head (izlazni sloj) nije vezan (tied) za embedding tabelu, za razliku od GPT-2/3 - projektuje nazad u rečnik nakon poslednjeg RMSNorm-a.
- Cross-entropy gubitak se računa nad spljoštenim $[B \times T, \text{vocab_size}]$ logits-ima u odnosu na sledeće tokene - ekvivalentno gubitku po svakoj poziciji.

Trening i inference

oracle/main.py - klasa Runner

Trening + inference

Uređaj: CUDA ako je dostupna, inače CPU · Optimizator: AdamW (lr=1e-3, wd=0.01)

- Nakon treninga se čuva `model.state_dict()` u `pico_gpt_oracle.pth`, koji se kasnije učitava za generisanje teksta.
- Inferencija radi kroz REPL petlju: korisnik unese prompt -> tokenizuje se -> `generate_text()` generiše tekst token po token, uzorkujući iz raspodele verovatnoća.
- Temperatura (podrazumevano 0.65) kontroliše nasumičnost: niže vrednosti → predvidljiviji tekst, više vrednosti → kreativniji, ali rizičniji izlaz.

Hiperparametri

<code>B (veličina batch-a)</code>	32
<code>T (dužina niza)</code>	128
<code>C (d_model)</code>	256
<code>L (n_layers)</code>	4
<code>NH (n_heads)</code>	8
<code>lr</code>	1e-3
<code>weight decay</code>	0.01
<code>max_steps</code>	2.000
<code>eval_interval</code>	200

Ključne dizajnerske odluke

Odabrane odluke i razlozi koji stoje iza njih

Aspekt	Odluka	Obrazloženje / korist
Tokenizacija	Nivo karaktera, fiksni rečnik	Potpuno razumljivo - nije potreban spoljni tokenizer
Pozicijsko kodiranje	RoPE (rotaciono)	Generalizacija na duže nizove, bez dodatnih parametara
Normalizacija	RMSNorm	Preskače oduzimanje srednje vrednosti; dokazano u modernim LLM-ovima
Aktivacija	SwiGLU (gated SiLU)	Bolja izražajnost uz sličan broj operacija
Strimovanje podataka	Učitavanje po delovima, povremena zamena	Ograničen RAM uz statističku raznovrsnost
Optimizator	AdamW, lr=1e-3, wd=0.01	Standardan, dobro regularizovan trening transformera

UKRATKO



PicoGPT je minimalan, open source dekodler-Transformer koji uči da predviđa sledeći karakter u TinyStories tekstovima. Pipeline strimuje tekst u delove fiksne veličine, gradi zajednički rečnik na nivou karaktera, i šalje mini-blokove ID-jeva tokena modelu.

Embedding

RMSNorm

GQA Attn + RoPE

RMSNorm

SwiGLU FFN

Final RMSNorm

LM Head

Trening koristi AdamW sa umerenom brzinom učenja, periodičnu evaluaciju gubitka i kratko generisanje teksta radi praćenja napretka. Kod je namerno modularan - tokenizer, chunker, batch loader, slojevi modela, trening petlja - tako da se svaki deo može proučavati, zameniti ili ponovo implementirati u čistom NumPy-ju, za edukaciju ili primenu na ograničenom hardveru.